

L02: Basic modeling



$st, du, vi, tk, gw, kl :: \{0,1\}$

$52*st + 85*du + 60*vi + 84*tk + 117*kl + 35*gw \leq 4*52$

$\text{maximize}(50*st + 80*du + 75*vi + 82*tk + 95*kl + 7*gw)$

Prof. Tias Guns and Dr. Hendrik ‘Henk’ Bierlee

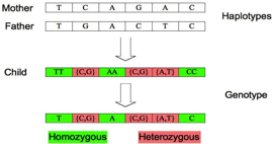
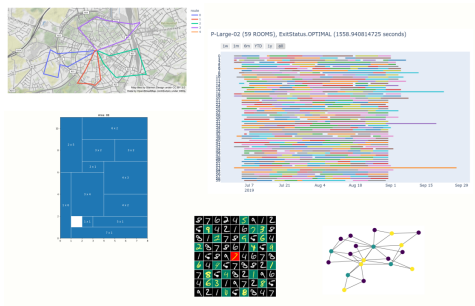
KU LEUVEN

Partly based on slides from Pierre Flener.

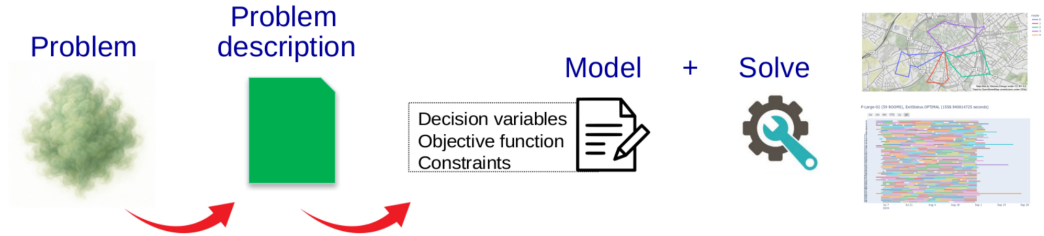
Outline

1. Recap: combinatorial problem to solution
2. Modeling basics
 - Basic example
 - Decision variables, domains, and constraints
 - Logical constraints
 - Integer constraints
 - Global constraints
 - Objective functions
3. Modeling with the CPMpy library
4. Solving the problem
5. Debugging

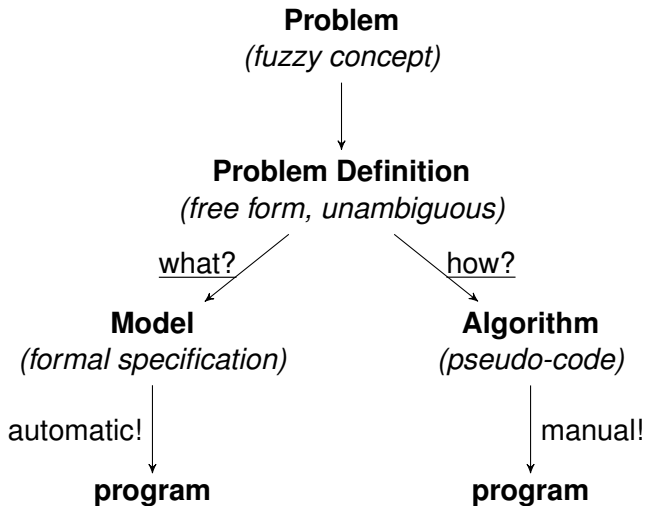
Combinatorial Optimisation



Declarative Model + Solve paradigm



Modeling (declarative) vs Programming (imperative)



Example (Sudoku)

8								
		3	6					
	7			9		2		
	5				7			
				4	5	7		
			1				3	
		1					6	8
		8	5				1	
	9					4		

8	1	2	7	5	3	6	4	9
9	4	3	6	8	2	1	7	5
6	7	5	4	9	1	2	8	3
1	5	4	2	3	7	8	9	6
3	6	9	8	4	5	7	2	1
2	8	7	1	6	9	5	3	4
5	2	1	9	7	4	3	6	8
4	3	8	5	2	6	9	1	7
7	9	6	3	1	8	4	5	2

The goal of Sudoku is to *complete* a 9x9 grid with numbers so that each row, column and 3x3 section contain each digit between 1 and 9 once.

8								
		3	6					
	7			9		2		
	5				7			
				4	5	7		
			1				3	
		1					6	8
		8	5				1	
	9					4		

8	1	2	7	5	3	6	4	9
9	4	3	6	8	2	1	7	5
6	7	5	4	9	1	2	8	3
1	5	4	2	3	7	8	9	6
3	6	9	8	4	5	7	2	1
2	8	7	1	6	9	5	3	4
5	2	1	9	7	4	3	6	8
4	3	8	5	2	6	9	1	7
7	9	6	3	1	8	4	5	2

Example (Sudoku CP model)

$$G_{ij} \in \{1, 2, \dots, 9\}$$

$$\text{ALLDIFFERENT}(\{G_{ij} \mid j \in [1..9]\})$$

$$\text{ALLDIFFERENT}(\{G_{ij} \mid i \in [1..9]\})$$

$$\text{ALLDIFFERENT}(\{G_{kl} \mid k \in [i..i+2], l \in [j..j+2]\})$$

$$G_{ij} = v$$

$$\forall i, j \in \{1, \dots, 9\}$$

$$\forall i \in \{1, \dots, 9\}$$

$$\forall j \in \{1, \dots, 9\}$$

$$\forall i, j \in \{1, 4, 7\}$$

$$\forall (i, j, v) \in \mathcal{D}$$

8								
		3	6					
	7			9		2		
	5				7			
				4	5	7		
			1				3	
		1					6	8
		8	5				1	
	9					4		

8	1	2	7	5	3	6	4	9
9	4	3	6	8	2	1	7	5
6	7	5	4	9	1	2	8	3
1	5	4	2	3	7	8	9	6
3	6	9	8	4	5	7	2	1
2	8	7	1	6	9	5	3	4
5	2	1	9	7	4	3	6	8
4	3	8	5	2	6	9	1	7
7	9	6	3	1	8	4	5	2

Example (Sudoku in CPMpy (indexing offset 0))

```

1 import cpmypy as cp
2 #given = np.array(...) # load the clues, uses '0' for the empty cells
3 grid = cp.intvar(1,9, shape=given.shape, name="grid") # Decision variables
4 model = cp.Model(
5     [cp.AllDifferent(row) for row in grid],
6     [cp.AllDifferent(col) for col in grid.T], # numpy's Transpose
7     [cp.AllDifferent(grid[i:i+3, j:j+3]) \
8         for i in range(0, 9, 3) for j in range(0, 9, 3)],
9     grid[given!=0] == given[given!=0], # enforce the clues
10 )
11 model.solve()

```

```

1 import cpmpy as cp
2 #given = np.array(...) # load the clues, uses '0' for the empty cells
3 grid = cp.intvar(1,9, shape=given.shape, name="grid") # Decision variables
4 model = cp.Model(
5     [cp.AllDifferent(row) for row in grid],
6     [cp.AllDifferent(col) for col in grid.T], # numpy's Transpose
7     [cp.AllDifferent(grid[i:i+3, j:j+3]) \
8         for i in range(0, 9, 3) for j in range(0, 9, 3)],
9     grid[given!=0] == given[given!=0], # enforce the clues
10 )
11 model.solve()

```

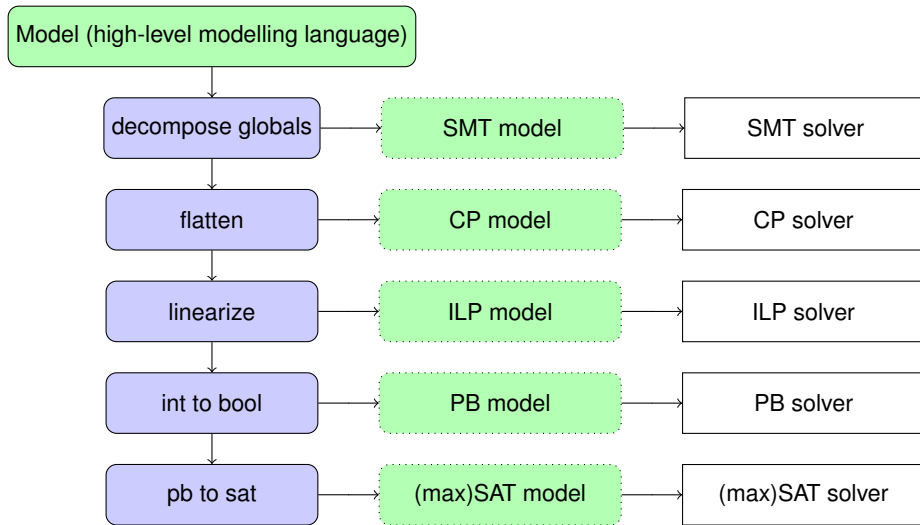
ExitStatus.FEASIBLE (0.008928985 seconds)

8*	1	2		7	5	3		6	4	9
9	4	3*		6*	8	2		1	7	5
6	7*	5		4	9*	1		2*	8	3
-----+-----+-----										
1	5*	4		2	3	7*		8	9	6
3	6	9		8	4*	5*		7*	2	1
2	8	7		1*	6	9		5	3*	4
-----+-----+-----										
5	2	1*		9	7	4		3	6*	8*
4	3	8*		5*	2	6		9	1*	7
7	9*	6		3	1	8		4*	5	2

Sudoku takeaways

- ▶ The Sudoku problem was formulated in a **declarative** manner: we specify **what** to solve, not **how** to solve
- ▶ High-level modeling languages support high-level constraints such as ALLDIFFERENT(...) (and more such as NOOVERLAP(...), ELEMENT(...), MIN(...), ...)
- ▶ The modeling language provides unified access to vastly different **solver technologies**, each with their own performance characteristics and capabilities
- ▶ Much engineering and research goes into making the modeling language **expressive** and the solver **fast**

From Model to Model to Solver: transformations



Outline

1. Recap: combinatorial problem to solution
2. Modeling basics
 - Basic example
 - Decision variables, domains, and constraints
 - Logical constraints
 - Integer constraints
 - Global constraints
 - Objective functions
3. Modeling with the CPMpy library
4. Solving the problem
5. Debugging

Outline

1. Recap: combinatorial problem to solution
2. Modeling basics
 - Basic example
 - Decision variables, domains, and constraints
 - Logical constraints
 - Integer constraints
 - Global constraints
 - Objective functions
3. Modeling with the CPMpy library
4. Solving the problem
5. Debugging

Tias' Belgian beer guide



Stella Artois, from Leuven, 5.2%, must-try factor: 5/10



Duvel, devilish blond, 8.5%, must-try factor: 8/10



Vedett IPA, tastefully hoppy, 6%, must-try factor: 7.5/10



Tripel Karmeliet, strong blond, 8.4%, must-try factor: 8.2/10



Gouden Carolus **Whiskey Infused**, 11.7%, must-try factor: 9.5/10



Kriek Lindemans, sweet cherry beer, 3.5%, must-try factor: 7/10

Belgian Beer Tasting problem

Problem description: Try at most 4 Stella's worth of %, but have the highest sum of must-try factors possible

Parameters What are the **knowns**?

Variables What are the **unknowns**?

Domains What are the **possible values** of the variables?

Constraints What are the **restrictions** between variables?

Objective Is there something to **min- or maximize**?

A mathematical description (i.e. a **model**):

Belgian Beer Tasting problem

Problem description: Try at most 4 Stella's worth of %, but have the highest sum of must-try factors possible

Parameters What are the **knowns**? Stella is 5.2% with must-try factor 5/10

Variables What are the **unknowns**?

Domains What are the **possible values** of the variables?

Constraints What are the **restrictions** between variables?

Objective Is there something to **min- or maximize**?

A mathematical description (i.e. a **model**):

Belgian Beer Tasting problem

Problem description: Try at most 4 Stella's worth of %, but have the highest sum of must-try factors possible

Parameters What are the **knowns**? Stella is 5.2% with must-try factor 5/10

Variables What are the **unknowns**? "Do we try a Stella?" = *st*

Domains What are the **possible values** of the variables?

Constraints What are the **restrictions** between variables?

Objective Is there something to **min- or maximize**?

A mathematical description (i.e. a *model*):

st, du, vi, tk, gc, kl

Belgian Beer Tasting problem

Problem description: Try at most 4 Stella's worth of %, but have the highest sum of must-try factors possible

Parameters What are the **knowns**? Stella is 5.2% with must-try factor 5/10

Variables What are the **unknowns**? "Do we try a Stella?" = st

Domains What are the **possible values** of the variables? $st \in \{0, 1\}$

Constraints What are the **restrictions** between variables?

Objective Is there something to **min- or maximize**?

A mathematical description (i.e. a **model**):

$$st, du, vi, tk, gc, kl \in \{0, 1\}$$

Belgian Beer Tasting problem

Problem description: Try at most 4 Stella's worth of %, but have the highest sum of must-try factors possible

Parameters What are the **knowns**? Stella is 5.2% with must-try factor 5/10

Variables What are the **unknowns**? "Do we try a Stella?" = st

Domains What are the **possible values** of the variables? $st \in \{0, 1\}$

Constraints What are the **restrictions** between variables? $52 * st + \dots \leq 4 * 52$

Objective Is there something to **min- or maximize**?

A mathematical description (i.e. a **model**):

$$st, du, vi, tk, gc, kl \in \{0, 1\}$$

$$52 * st + 85 * du + 60 * vi + 84 * tk + 117 * gc + 35 * kl \leq 4 * 52$$

Belgian Beer Tasting problem

Problem description: Try at most 4 Stella's worth of %, but have the highest sum of must-try factors possible

Parameters What are the **knowns**? Stella is 5.2% with must-try factor 5/10

Variables What are the **unknowns**? "Do we try a Stella?" = *st*

Domains What are the **possible values** of the variables? $st \in \{0, 1\}$

Constraints What are the **restrictions** between variables? $52 * st + \dots \leq 4 * 52$

Objective Is there something to **min- or maximize**? max. $50 * st + 80 * du + \dots$

A mathematical description (i.e. a **model**):

$$st, du, vi, tk, gc, kl \in \{0, 1\}$$

$$52 * st + 85 * du + 60 * vi + 84 * tk + 117 * gc + 35 * kl \leq 4 * 52$$

$$\text{maximize } 50 * st + 80 * du + 75 * vi + 82 * tk + 95 * gc + 70 * kl$$

```

1 import cpmPy as cp
2 m = cp.Model()
3
4 # Create six (named) Boolean variables, where 'X[i]' is True iff we try beer 'i'
5 X = cp.boolvar(shape=6, name=["Stella", "Duvel", "Vedett", "Karmeliet", "Carolus",
    "Kriek"])
6 st, du, vi, tk, gc, kl = X # "Unpack" the variables
7
8 # Constraint: Try less than 4 Stella's worth
9 m.add(52*st + 85*du + 60*vi + 84*tk + 117*gc + 35*kl <= 4*52)
10 # Objective: Maximize the must-try factor
11 m.maximize(50*st + 80*du + 75*vi + 82*tk + 95*gc + 70*kl)
12
13 m.solve()
14 print(m.status())
15 print(", ".join(str(x) for x in X if x.value() is True), f"({m.objective_value()})")
    ")

```

T02_beer_tasting.py

```

ExitStatus.OPTIMAL (0.001963841 seconds)
Duvel, Karmeliet, Kriek (232)

```

> python T02_beer_tasting.py

Outline

1. Recap: combinatorial problem to solution
2. Modeling basics
 - Basic example
 - Decision variables, domains, and constraints**
 - Logical constraints
 - Integer constraints
 - Global constraints
 - Objective functions
3. Modeling with the CPMpy library
4. Solving the problem
5. Debugging

Decision variables

Decision variables are the **unknowns** of the problem. Each decision variable has a domain (i.e. a set of values).

The two primary types of variables are:

- ▶ Boolean variables, e.g. $st \in \{0, 1\}$ (whether we try a Stella or not)
- ▶ Integer variables, e.g. $st \in [0..2]$ (whether we try 0, 1, or 2 Stella's)

An assignment maps each variable to a value from its domain (e.g. $st = 1, du = 0, \dots$)

The minimum value of the domain is called the lower bound, the maximum value the upper bound

In this course, we only consider domains which are finite (no unbounded, e.g. $st \in [0..]$) and discrete (no floating point, e.g. $st \in [0.0..2.5]$). Different modeling languages support different variable types (e.g. floats, sets, strings, bitvectors, ...)

Expressions, constraints, and solutions

A model contains **expressions**, i.e. a tree of mathematical symbols. A constraint is a **relation** over variables, i.e. an expression which evaluates to *true* or *false*. For a feasible assignment (i.e. **solution**), all constraints evaluate to *true*.

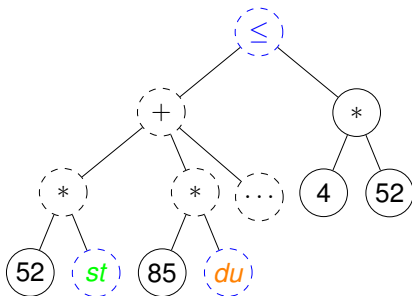


Figure: An expression tree for $52 * st + 85 * du + \dots \leq 4 * 52$. Each node is itself an expression. Known expressions (parameters) are in solid nodes, unknown expressions (variables) in dashed nodes. Boolean-valued expressions are in blue, integer-valued expressions in black.

Outline

1. Recap: combinatorial problem to solution
2. Modeling basics
 - Basic example
 - Decision variables, domains, and constraints
 - Logical constraints**
 - Integer constraints
 - Global constraints
 - Objective functions
3. Modeling with the CPMpy library
4. Solving the problem
5. Debugging

Logical constraints

Logical constraints involve Boolean operators over Boolean-valued expressions

Negation, not: $\neg p$

Or, disjunction: $p \vee q$

And, conjunction: $p \wedge q$

Implication, conditional: $p \rightarrow q$

Equivalence, bi-conditional, if-and-only-if: $p \leftrightarrow q$, or $p == q$

Exclusive-or, xor: $p \oplus q$

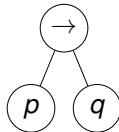


Figure: An expression tree for $p \rightarrow q$ containing **three** expressions

- ▶ E.g. “We’re out of Kriek!”: $\neg \textcolor{red}{kl}$
 - ▶ Disallows the old optimal solution, Duvel, Karmeliet, Kriek (232)
 - ▶ New optimal solution: Stella, Vedett, Karmeliet (207)
- ▶ E.g. “If I try Stella, I should not try Duvel”: $\textcolor{green}{st} \rightarrow \neg \textcolor{orange}{du}$

When does a logical constraint evaluate to *true*?

Each operator is defined by its **truth table**

p	q	$\neg q$	$p \vee q$	$p \wedge q$	$p \leftrightarrow q$	$p \oplus q$	$p \rightarrow q$
F	F	T	F	F	T	F	
F	T	F	T	F	F	T	
T	F	...	T	F	F	T	
T	T	...	T	T	T	F	

When does a logical constraint evaluate to *true*?

Each operator is defined by its **truth table**

p	q	$\neg q$	$p \vee q$	$p \wedge q$	$p \leftrightarrow q$	$p \oplus q$	$p \rightarrow q$
F	F	T	F	F	T	F	T
F	T	F	T	F	F	T	
T	F	...	T	F	F	T	
T	T	...	T	T	T	F	

When does a logical constraint evaluate to *true*?

Each operator is defined by its **truth table**

p	q	$\neg q$	$p \vee q$	$p \wedge q$	$p \leftrightarrow q$	$p \oplus q$	$p \rightarrow q$
F	F	T	F	F	T	F	T
F	T	F	T	F	F	T	T
T	F	...	T	F	F	T	
T	T	...	T	T	T	F	

When does a logical constraint evaluate to *true*?

Each operator is defined by its **truth table**

p	q	$\neg q$	$p \vee q$	$p \wedge q$	$p \leftrightarrow q$	$p \oplus q$	$p \rightarrow q$
F	F	T	F	F	T	F	T
F	T	F	T	F	F	T	T
T	F	...	T	F	F	T	F
T	T	...	T	T	T	F	

When does a logical constraint evaluate to *true*?

Each operator is defined by its **truth table**

p	q	$\neg q$	$p \vee q$	$p \wedge q$	$p \leftrightarrow q$	$p \oplus q$	$p \rightarrow q$
F	F	T	F	F	T	F	T
F	T	F	T	F	F	T	T
T	F	...	T	F	F	T	F
T	T	...	T	T	T	F	T

When does a logical constraint evaluate to *true*?

Each operator is defined by its **truth table**

p	q	$\neg q$	$p \vee q$	$p \wedge q$	$p \leftrightarrow q$	$p \oplus q$	$p \rightarrow q$
F	F	T	F	F	T	F	T
F	T	F	T	F	F	T	T
T	F	...	T	F	F	T	F
T	T	...	T	T	T	F	T

Boolean-valued expressions are equivalent if they have the same truth table. We can rewrite equivalent expressions (using $\equiv$ notation):

- ▶ $p \rightarrow q \equiv \neg p \vee q$
- ▶ $p \leftrightarrow q \equiv (p \rightarrow q) \wedge (q \rightarrow p)$
- ▶ $p \oplus q \equiv p \leftrightarrow \neg q \equiv p \neq q$

Quantification

Unary operators have one argument, binary operators have two, n -ary have n .

The Boolean quantifiers are the n -ary generalizations of $\wedge$ and $\vee$:

- ▶ Universal quantification, i.e. “for all”: $\forall_{x_i \in X} x_i \equiv x_1 \wedge x_2 \wedge \dots$
- ▶ Existential quantification, i.e. “there is/exists”: $\exists_{x_i \in X} x_i \equiv x_1 \vee x_2 \vee \dots$

Outline

1. Recap: combinatorial problem to solution
2. Modeling basics
 - Basic example
 - Decision variables, domains, and constraints
 - Logical constraints
 - Integer constraints**
 - Global constraints
 - Objective functions
3. Modeling with the CPMpy library
4. Solving the problem
5. Debugging

Simple comparison constraints

Simple comparisons ($=$, $\neq$, $<$, $\leq$, $>$, $\geq$) over an integer variable, e.g. $x \in [1..5]$:

$x = 3$ solution(s)

$x < 4$ solution(s)

$x \neq 6$ solution(s)

$x = 7$ solution(s)

Simple comparison constraints

Simple comparisons ($=$, $\neq$, $<$, $\leq$, $>$, $\geq$) over an integer variable, e.g. $x \in [1..5]$:

$x = 3$	1 solution(s)	$\{x = 3\}$
$x < 4$	solution(s)	
$x \neq 6$	solution(s)	
$x = 7$	solution(s)	

Simple comparison constraints

Simple comparisons ($=$, $\neq$, $<$, $\leq$, $>$, $\geq$) over an integer variable, e.g. $x \in [1..5]$:

$x = 3$	1 solution(s)	$\{x = 3\}$
$x < 4$	3 solution(s)	$\{x = 1, x = 2, x = 3\}$
$x \neq 6$	solution(s)	
$x = 7$	solution(s)	

Simple comparison constraints

Simple comparisons ($=, \neq, <, \leq, >, \geq$) over an integer variable, e.g. $x \in [1..5]$:

$x = 3$	1 solution(s)	$\{x = 3\}$
$x < 4$	3 solution(s)	$\{x = 1, x = 2, x = 3\}$
$x \neq 6$	5 solution(s)	$\{x = 1, \dots, x = 5\}$
$x = 7$	solution(s)	

Simple comparison constraints

Simple comparisons ($=, \neq, <, \leq, >, \geq$) over an integer variable, e.g. $x \in [1..5]$:

$x = 3$	1 solution(s)	$\{x = 3\}$
$x < 4$	3 solution(s)	$\{x = 1, x = 2, x = 3\}$
$x \neq 6$	5 solution(s)	$\{x = 1, \dots, x = 5\}$
$x = 7$	0 solution(s)	$\emptyset$

Simple comparison constraints

Simple comparisons ($=$, $\neq$, $<$, $\leq$, $>$, $\geq$) over an integer variable, e.g. $x \in [1..5]$:

$x = 3$	1 solution(s)	$\{x = 3\}$
$x < 4$	3 solution(s)	$\{x = 1, x = 2, x = 3\}$
$x \neq 6$	5 solution(s)	$\{x = 1, \dots, x = 5\}$
$x = 7$	0 solution(s)	$\emptyset$

Note that while these expressions involve integer variables, they still evaluate to Booleans. So, as Boolean-valued expressions, they can be used with Boolean operators:

$$(x = 3) \wedge (y > 5)$$
$$(y \neq 6) \rightarrow (a \vee (z < 2))$$

Tip: good use of brackets (...) avoids ambiguity.

Arithmetic constraints

Arithmetic constraints combine arithmetic operations (+, −, *, /) over integers with a comparisons (=, ≠, <, ≤, >, ≥), e.g. for $x, y \in [1..5]$:

$$x + y = 3$$

$$x < y$$

$$y - z \neq 6$$

$$2 * z - (x * y + y) < 2$$

Arithmetic constraints

Arithmetic constraints combine arithmetic operations (+, −, *, /) over integers with a comparisons (=, ≠, <, ≤, >, ≥), e.g. for $x, y \in [1..5]$:

$$x + y = 3$$

$$x < y$$

$$y - z \neq 6$$

$$2 * z - (x * y + y) < 2$$

Integer division $x/y = 5$ is tricky because it is **undefined** in assignments where $y = 0$. Division is a **partial function**. Some languages/systems simply forbid a division where the denominator has 0 in its domain. There are more peculiarities with integer division, such as using *floor division* or *rounding division* for negative numbers.

Integer linear constraints

A **linear term** is a multiplication of a variable with a constant, e.g. $2 * x$. A **linear expression** is a sum of linear terms and a constant, e.g. $2 * x - 3 * y - 6$

A **linear constraint** is a comparison between two linear expressions, e.g. $2 * x - 3 * y - 6 \leq 5 * z$. A **linear inequality** uses only the comparisons ($<$, $\leq$, $>$, $\geq$).

Linear inequalities can be rewritten to a normal form with integers a_i , k and integer variables x_i :

$$a_1 * x_1 + \dots + a_n * x_n \leq k \quad (\text{e.g. } 2 * x - 3 * y - z \leq 6)$$

We can also normalize **linear equality** and **disequality constraints**:

- ▶ $x + y = 5 \equiv (x + y \leq 5) \wedge (x + y \geq 5)$
- ▶ $x + y \neq 5 \equiv (x + y < 5) \vee (x + y > 5)$, however, we require additional variables to resolve the disjunction, $\vee$

Arithmetic constraints

Many other arithmetic operations exist:

- ▶ Absolute value $|a|$
- ▶ Modulo $x \% y$
- ▶ Minimum $\min(x, y, z)$
- ▶ Maximum $\max(x, y, z)$

These can be used in arithmetic constraints too (e.g. arithmetic operators + comparison).

Just like simple constraints, arithmetic constraints are Boolean-valued expressions too; and can be used in Boolean-valued expressions.

A **nested expression** nests all sorts of Boolean operators and/or arithmetic constraints and operators. A contrived example:

$$(a \vee (|x - y| > z/2)) \rightarrow (r * s \neq \max(x, y - t, |z - 3|)) \wedge (b \otimes (c \leftrightarrow d))$$

Outline

1. Recap: combinatorial problem to solution
2. Modeling basics
 - Basic example
 - Decision variables, domains, and constraints
 - Logical constraints
 - Integer constraints
 - Global constraints**
 - Objective functions
3. Modeling with the CPMpy library
4. Solving the problem
5. Debugging

Global constraints

A **global constraint** is a named constraint that captures a non-trivial relationship between decision variables (common combinatorial substructure). Often, there is no equivalent operation in standard mathematics, though they frequently appear in combinatorial problems. E.g.:

- ▶ `ALLDIFFERENT(x, y, z)`
- ▶ `CIRCUIT([x, y, z])`
- ▶ `TABLE([x, y], [[1, 2], [1, 4], [3, 4]])`
- ▶ ...

The globals are the subject of the next lecture

Outline

1. Recap: combinatorial problem to solution
2. Modeling basics
 - Basic example
 - Decision variables, domains, and constraints
 - Logical constraints
 - Integer constraints
 - Global constraints
 - Objective functions**
3. Modeling with the CPMpy library
4. Solving the problem
5. Debugging

Objective functions

The **objective function** is an expression that we aim to minimize or maximize (e.g. the sum of must-try factors). The objective does not remove solutions, but instead **ranks** them.

$$x \in [1..5], x \leq 3, \text{ maximize } x$$

This model still has 3 solutions, but one is **optimal**.

The objective can be any expression, including global functions or other constraints. A **hard constraint** can be modeled as a **soft constraint** through the objective.

E.g. changing the above model, we prefer a solution where x is even, but this may be violated. *How many solutions and (optimal) solutions are there now?*

$$x \in [1..5], x \leq 3, \text{ maximize } x + \llbracket x \% 2 = 0 \rrbracket$$

Outline

1. Recap: combinatorial problem to solution
2. Modeling basics
 - Basic example
 - Decision variables, domains, and constraints
 - Logical constraints
 - Integer constraints
 - Global constraints
 - Objective functions
3. Modeling with the CPMpy library
4. Solving the problem
5. Debugging

Using CPMpy includes the following:

- ▶ Import and model creation
- ▶ Decision variables
- ▶ Constraints
- ▶ Objective function (optional)
- ▶ Solving
- ▶ Printing output

Example (Showcase)

```
1 import cpmPy as cp
2 m = cp.Model()
3
4 # Decision variables
5 b = cp.boolvar(name="b")
6 X = cp.intvar(1,10, shape=3, name="X")
7
8 # Constraints
9 m.add(X[0] == 1)
10 m.add(cp.AllDifferent(X))
11 m.add(b.implies(X[1] + X[2] > 5))
12
13 # Objective function (optional)
14 m.maximize(cp.sum(X) + 100*b)
15
16 if m.solve():
17     print(X.value(), b.value())
18     print("obj:", m.objective_value())
19 else:
20     print("No solution found.")
21     print(m)
```

Install, import, create model

Single page, all you need documentation:

<https://cumpy.readthedocs.io/en/latest/modeling.html>

Installing: `pip install cumpy`

Importing and model creation:

```
1 import cumpy as cp
2 m = cp.Model()
```

You can also `from cumpy import *` which will override any/all/min/max/sum for convenience but this can be confusing for novices (e.g. when does `sum` compute a value or create an expression).

Decision variables

CPMpy supports **discrete decision variables**, namely Boolean and integer decision variables:

```
1 b = cp.boolvar(name="b")
2 x = cp.intvar(lb=1,ub=10, name="x")
```

Variables have a **domain**, a finite set of allowed values:

- ▶ for Boolean variables, this is implicitly $\{0, 1\}$
- ▶ for integer variables, this is specified as the first two parameters:
lb=lower bound, *ub*=upper bound, domain: [*lb..ub*]

If you want a **sparse domain**, containing only a few values, you can:

- ▶ Add constraints to forbid specific values, e.g. $x \neq 3$, $x \neq 5$, $x \neq 7$
- ▶ Or use the shorthand *InDomain* global constraint:

```
cp.InDomain(x, [1,2,4,6,8,9])
```

Decision variables 2/2

Decision variables have a unique **name**. You can set it yourself, otherwise a unique name will automatically be assigned to it. If you print decision variables, `print(b, x)`, it will print the name.

CPMpy creates *n-dimensional NumPy arrays* when creating variables! This is very convenient for NumPy-style *vectorized* operations and for integration with machine learning libraries.

The *shape* argument allows you to specify the dimensions of the array. All variables will have the same initial domain:

```
1 B = cp.boolvar(shape=4, name="B")
2 print(B)  # [B[0] B[1] B[2] B[3]]
3
4 X = cp.intvar(1,10, shape=(2,2), name="X")
5 print(X)  # [[X[0,0] X[0,1]]
6           # [X[1,0] X[1,1]]]
```

Note: NumPy indexing

Python creates lists of lists, each requiring an index:

```
1 A = [ ["00", "01"],  
2       ["10", "11"]]  
3 print(A[0][1]) # out: '01'
```

NumPy creates an array, which allows indexing with a tuple:

```
1 B = np.array(A)  
2 print(B[0,1]) # out: '01'
```

Also accepts ':' which means this entire dimension:

```
1 print(B[0,:]) # out: ['00' '01']
```

Or using an equal sized array of Booleans (also called a 'selector'):

```
1 Sel = [[False, True],  
2        [True, False]]  
3 print(B[Sel]) # out: ['01' '10']
```

Advanced: Vectorized operations

Because decision variables are NumPy arrays in CPMpy, you can also do **vectorized operations** on them: an operation on two equal sized arrays will create an (equal sized) array of element-wise operations:

Example (vectorized operations)

```
1 X = cp.intvar(1,9, shape=3, name="X")
2 A = [1,2,4]
3
4 print(X == A) # output: [X[0]==1 X[1]==2 X[2]==4]
```

Broadcasting in NumPy allows operations between arrays of different shapes, by automatically expanding the smaller array along its dimensions to match the larger array's shape:

Example (broadcasting)

```
1 print(X == 3) # output: [X[0]==3 X[1]==3 X[2]==3]
```

Expressing constraints

A **constraint** is a Boolean-valued expression that is added to a model. The solver will enforce it to always be true, e.g.:

```
1 m.add(X[0] == 1)
2 m.add(cp.AllDifferent(X))
3 m.add(b.implies(X[1] + X[2] > 5))
```

The `m += o` is Python syntactic sugar for `m.__add__(o)`. Alternatively, you can use `m.add(o)`.

We will differentiate *Boolean-valued expressions* like `X[0] == 1` and *integer-valued expressions* like `X[1] + X[2]`. Only Boolean-valued expressions can be added as a constraint.

Common constraints

Example (Typical logical constraints)

```
1 (a,b,c) = cp.boolvar(shape=3)
2 m.add(a | b)    # a OR b
3 m.add(~(a & b))  # NOT (a AND b)
4 m.add(a.implies(b | c))  # a -> (b OR c)
5 m.add(a == b)   # equivalence: (a -> b) & (b -> a)
6 m.add(a != b)   # same as ~(a==b) and same as (a == ~b)
```

CPMpy overloads the Python bitwise operators `&`, `|`, `~`. They have precedence over all other operators, so `a == 0 | b == 1` is **wrongly** interpreted as `a == (0 | b) == 1` -- WRONG!. So make sure to **always write explicit brackets** to express `(a == 0) | (b == 1)`.

Example (*n*-ary logical constraints)

```
1 Bv = cp.boolvar(shape=3)
2 m.add(cp.any([Bv[0], Bv[1], Bv[2]]))  # explicit list
3 m.add(~cp.all(Bv))  # (numpy) array
```

Common constraints 2/4

Example (Typical comparison constraints)

```
1 b = cp.boolvar()
2 x = cp.intvar(0, 10)
3 Iv = cp.intvar(0, 10, shape=3)
4
5 m.add(x > 3)
6 m.add(x != 6)
7
8 m.add(Iv == 1)  # vectorized, shorthand for:
9 m.add([Iv[0] == 1, Iv[1] == 1, Iv[2] == 1])
```

You can not use a numeric expression as a Boolean-valued expression, e.g.

invalid: $b \mid x \text{ -- WRONG!}$, you need to add your intended meaning of truth:

$b \mid (x \neq 0)$

Common constraints 3/4

Example (Some arithmetic constraints)

```
1 Xs = cp.intvar(0, 10, shape=3, name="Xs")
2 Ys = cp.intvar(1, 10, shape=3, name="Ys")
3 W = np.array([1,3,-5]) # numpy array for use in vectorized multiplication
4
5 m.add(Xs[0] - Ys[0] == 5)
6 m.add(cp.sum(Xs) != 1)
7 m.add(cp.sum(W*Xs) > 3) # 1*Xs[0] + 3*Xs[1] + (-5)*Xs[2] > 3
8
9 # arbitrary nested expressions:
10 m.add(3*Xs[0] < abs(5 - cp.max(Xs) + cp.min(Ys)))
```

You **can** use any Boolean-valued expression as a numeric expression, e.g. valid:

$b + x > 2$

Common constraints 4/4

Example (Typical global constraints)

```
1 Ivs = cp.intvar(1,10, shape=4, name="ivs")
2 b = cp.boolvar()
3
4 m.add(cp.AllDifferent(Ivs))
5 m.add(b.implies(cp.AllEqual(Ivs)))
6 m.add(cp.max(Ivs) == 3) # maximum of 'ivs' values equals 3
7 m.add(cp.Table(Ivs, [[1,1,2,4], # values must match one of the rows
8                        [1,2,3,6],
9                        [2,3,5,10]]))
```

Many more global constraints exist! We will see them throughout the course.

Handy summary sheet: <https://cumpy.readthedocs.io/en/latest/summary.html>

Objective functions (optional)

If a model has *no objective function* specified, then it is a **satisfaction problem**: the goal is to find out whether a solution, any solution, exists.

When an objective function is added it is an **optimisation problem** and this function needs to be minimized or maximized.

Example (Objective function, from showcase example)

```
1 # Objective function (optional)
2 m.maximize(cp.sum(X) + 100*b)
```

Any expression can be added as an objective function (**maximize()** or **minimize()**)

CPMpy does not support multi-objective optimisation yet:
multiple objective functions must either be aggregated into a weighted sum,
or handled outside the model.

Solving and printing output

Example (Solving)

```
1 Xs = cp.intvar(1,10, shape=3)
2 m = cp.Model( cp.AllDifferent(Xs) )
3 m.maximize(cp.sum(Xs))
4
5 hassol = m.solve()
```

`solve()` accepts arguments such as `time_limit=`, `solver=`, and solver-specific ones

Example (Printing output)

```
1 print("Status:", m.status()) # Status: ExitStatus.OPTIMAL (0.03033301 seconds)
2 if hassol:
3     print(m.objective_value(), Xs.value()) # 27 [10 9 8]
4 else:
5     print("No solution found.")
6     print(m) # pretty-prints the constraints in the model
```

Focus point: reification

Definition (Reification)

Reification replaces a constraint C (or any other Boolean-valued expression) for a Boolean **reification** variable $b \in \{0, 1\}$, and adds the constraint $b \leftrightarrow C$ to the model.

Example

Earlier, we rewrote the normalized linear disequality $x + y \neq 5$ to a disjunction. We can rewrite the disjunction by introducing reification variables $p, q \in \{0, 1\}$:

$$(x + y < 5) \vee (x + y > 5) \equiv$$

Focus point: reification

Definition (Reification)

Reification replaces a constraint C (or any other Boolean-valued expression) for a Boolean **reification** variable $b \in \{0, 1\}$, and adds the constraint $b \leftrightarrow C$ to the model.

Example

Earlier, we rewrote the normalized linear disequality $x + y \neq 5$ to a disjunction. We can rewrite the disjunction by introducing reification variables $p, q \in \{0, 1\}$:

$$(x + y < 5) \vee (x + y > 5) \equiv (p \vee q)$$

Focus point: reification

Definition (Reification)

Reification replaces a constraint C (or any other Boolean-valued expression) for a Boolean **reification** variable $b \in \{0, 1\}$, and adds the constraint $b \leftrightarrow C$ to the model.

Example

Earlier, we rewrote the normalized linear disequality $x + y \neq 5$ to a disjunction. We can rewrite the disjunction by introducing reification variables $p, q \in \{0, 1\}$:

$$(x + y < 5) \vee (x + y > 5) \equiv (p \vee q) \wedge (p \leftrightarrow (x + y < 5))$$

Focus point: reification

Definition (Reification)

Reification replaces a constraint C (or any other Boolean-valued expression) for a Boolean **reification** variable $b \in \{0, 1\}$, and adds the constraint $b \leftrightarrow C$ to the model.

Example

Earlier, we rewrote the normalized linear disequality $x + y \neq 5$ to a disjunction. We can rewrite the disjunction by introducing reification variables $p, q \in \{0, 1\}$:

$$(x + y < 5) \vee (x + y > 5) \equiv (p \vee q) \wedge (p \leftrightarrow (x + y < 5)) \wedge (q \leftrightarrow (x + y > 5))$$

One step closer to a constraint which is supported by the solver!

Reification enables high-level modelling (e.g. nested expressions), and efficient reuse of Boolean-valued expressions through their reification variable

General-purpose Modeling Languages

- ▶ CPMpy: <https://cpmpy.readthedocs.io/>
- ▶ MiniZinc: <https://www.minizinc.org>
- ▶ Essence and Essence': <https://constraintmodelling.org>
- ▶ OPL: <https://www.ibm.com/optimization-modeling>
- ▶ SMT-lib: <https://smtlib.cs.uiowa.edu>
- ▶ AIMMS: <https://aimms.com>
- ▶ AMPL: <https://ampl.com>
- ▶ GAMS: <https://gams.com>
- ▶ FICO Xpress Insight:
<https://www.fico.com/en/products/fico-xpress-optimization>
- ▶ Comet: <https://mitpress.mit.edu/books/constraint-based-local-search>
- ▶ ...

Outline

1. Recap: combinatorial problem to solution
2. Modeling basics
 - Basic example
 - Decision variables, domains, and constraints
 - Logical constraints
 - Integer constraints
 - Global constraints
 - Objective functions
3. Modeling with the CPMpy library
4. Solving the problem
5. Debugging

The contract between Modeler and Solver

- ▶ Each solver works in its own way, but CPMpy allows you to use all solvers in (mostly) the same way
- ▶ Given a model, the solver will (eventually) return a solution if it exists
 - ▶ There is no guarantee on *which* solution it finds (first)
- ▶ Solvers can find multiple solutions, but there is no guarantee on the order
 - ▶ In the case of optimization, most solvers will only output *intermediate* solutions which improve the objective
- ▶ Most (but not all) solvers use *exact* algorithms: they can find ***all*** solutions, or the ***optimal*** solution
 - ▶ Consequently, a solver can conclude a problem is *unsatisfiable* (i.e. has no solutions)
 - ▶ The search space is *huge*, e.g. d^n for n variables with domains of size d
 - ▶ Still, the solver apparently *exhaustively explores* the full search space!

1								
		3	6					
	7			9		2		
	5				7			
				4	5	7		
			1				3	
		1					6	8
		8	5				1	
	9					4		

```

1  import cpmPy as cp
2  #given = np.array(...) # load the clues, uses '0' for the empty cells
3  grid = cp.intvar(1,9, shape=given.shape, name="grid") # Decision variables
4  model = cp.Model(
5      [cp.AllDifferent(row) for row in grid],
6      [cp.AllDifferent(col) for col in grid.T], # numpy's Transpose
7      [cp.AllDifferent(grid[i:i+3, j:j+3]) \
8          for i in range(0, 9, 3) for j in range(0, 9, 3)],
9      grid[given!=0] == given[given!=0], # enforce the clues
10 )
11 model.solve()
12 print(model.status())

```

Solving combinatorial problems

Combinatorial problems: Huge **search space**

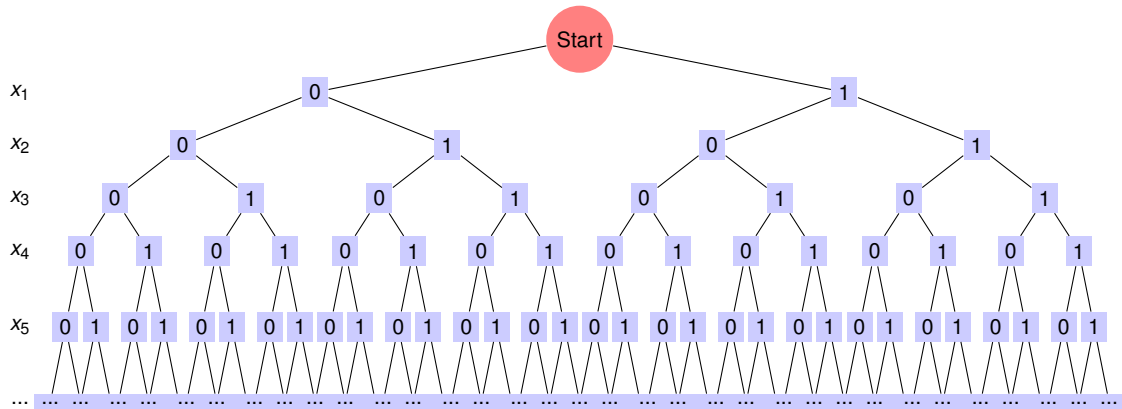


Figure: Reasoning and search for model $x_1, x_2, \dots \in \{0, 1\}, x_2 \neq x_3, x_1 \rightarrow \neg x_2, \dots$

Solving combinatorial problems

Combinatorial problems: Huge **search space** **needs intelligent search!**

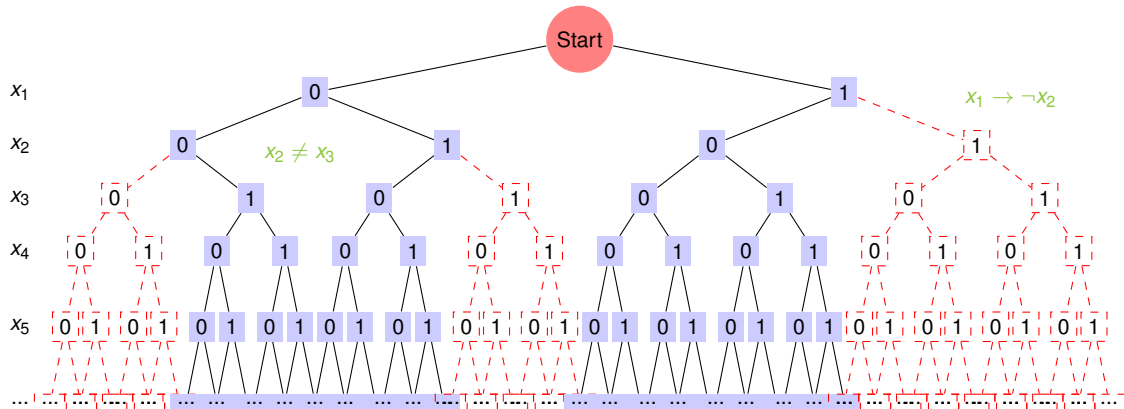


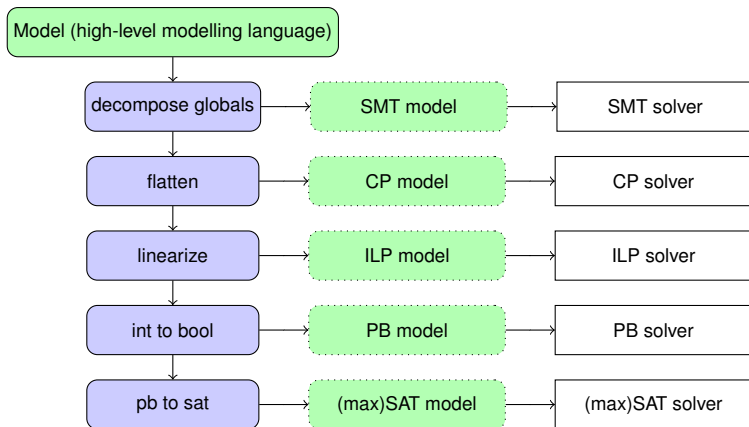
Figure: Reasoning and search for model $x_1, x_2, \dots \in \{0, 1\}, x_2 \neq x_3, x_1 \rightarrow \neg x_2, \dots$

Solving combinatorial problems

Different solvers/solving technologies can be used for that. They differ in:

- ▶ The supported types of domains and constraints
- ▶ How they perform search and propagation (CP vs MIP vs PB vs SAT)
- ▶ How they guide the search (heuristics, hyper-parameters)
- ▶ What capabilities are supported:
 - ▶ Exact algorithms versus non-exact
 - ▶ Proving unsatisfiability/optimality versus simply reporting it
 - ▶ Incremental solving i.e. allow constraints to be added/removed in-between solve calls

Model Transformations in CPMpy



Example solvers:

- ▶ SMT: Z3
- ▶ CP: OR-Tools, Choco, GCS, Minizinc (modeling language)
- ▶ ILP: Gurobi, CPLEX
- ▶ PB: Exact
- ▶ SAT: PySAT, Pindakaas (i.e. libraries which use SAT solvers Cadical, Kissat, ...)

Solving in CPMpy

We simply specify the solver to use (if installed):

```
1 model.solve("choco") # requires 'pip install cpmPy[choco]`  
2 model.solve("gurobi") # requires 'pip install cpmPy[gurobi]`  
3 cp.SolverLookup.solvernames() # return all locally installed solvers
```

Rather than just solving problems, we can make new algorithms which call solvers in creative ways:

```
solutions = 0 # initialize  
solution_limit = 5 # find 5 solutions  
  
# model.solve() returns true if a solution is found  
while model.solve() and solutions < solution_limit:  
    solutions += 1  
    # Constraint to enforce different solution  
    model.add(~cp.all(grid == grid.value()))
```

This meta-algorithm enumerates solutions by “blocking” each newly found solution, and resolving. This is what `model.solveAll()` does if the solver does not natively support solution enumeration

Outline

1. Recap: combinatorial problem to solution
2. Modeling basics
 - Basic example
 - Decision variables, domains, and constraints
 - Logical constraints
 - Integer constraints
 - Global constraints
 - Objective functions
3. Modeling with the CPMpy library
4. Solving the problem
5. Debugging

Debugging

You solve the problem, but
you get an **error**,
or no error, but also **no (correct) solution...**
Annoying, you have a **bug**.

How do you **debug** a model?



Debugging

General advise for debugging when modeling from expert modeller **Håkan Kjellerstrand**:

- ▶ Test the model **early and often**. This makes it easier to detect problems in the model.
- ▶ When a model is not working, **activate the constraints one by one** (e.g. comment out the other constraints) to test which constraint is the culprit:
 for each constraint c in *Constraints* **do**
 print “Trying:”, c
 Solve c
- ▶ **Check the domains** (see lower). The domains should be as small as possible, but not smaller. If they are too large it can take a lot of time to get a solution. If they are too small, then there will be no solution.

Debugging – CPMpy

General advise for debugging when modeling from expert modeller **Håkan Kjellerstrand**:

- ▶ Test the model **early and often**. This makes it easier to detect problems in the model.
- ▶ When a model is not working, **activate the constraints one by one** (e.g. comment out the other constraints) to test which constraint is the culprit.

```
1     for c in model.constraints:
2         print("Trying",c)
3         cp.Model(c).solve()
```

- ▶ **Check the domains** (see lower). The domains should be as small as possible, but not smaller. If they are too large it can take a lot of time to get a solution. If they are too small, then there will be no solution.

Debugging

The bug can be situated in one of three layers:

1. your model
2. the modeling library (CPMpy)
3. the solver

Ordered from most likely to least likely!

Bug in the solver

You try with the default solver (or another one) and you get an error, or not the desired solution.

Use a different solver and observe:

1. Outcome changes! It was a (rare) solver bug. Report it to the bug tracker of the modeling library or directly to the solver developers!
2. Outcome is the same! Not a solver error after all ...

Debugging a modeling error – CPMpy

You get an error when you create an expression?

Quirks in Python/CPMpy (from last lecture):

1. **Logical and/or:** Use `&` and `|`, and make sure to always put the subexpressions in brackets.

Example

write `(x == 1) & (y == 0)` instead of `x == 1 & y == 0`. The latter won't work. Python will think you meant `x == (1 & y) == 0`.

2. you can write `vars_list[other_var]` but you can't write `non_var_list[a_var]`. That is because the vars list knows CPMpy, and the non_var_list does not. Wrap it:
`non_var_list = cp.cpm_array(non_var_list)` first.
3. CPMpy overloads `all/any/max/min/sum/abs` to create expressions with them. Always use `cp.sum(v)` instead of `sum(v)`. You can also use directly NumPy's `v.sum()` instead, if `v` is a matrix or tensor.

Debugging a modeling error – CPMpy

You get an error when you create an expression... but you do not know why!

Print the constraints you create (or the sub-expressions), and check that the output matches what you wish to express!

Example

The following:

```
1 x = cp.intvar(0,5,shape=(2,2))
2 con = sum(x)
3 print(con)
```

will print [(IV0) + (IV2) (IV1) + (IV3)] and you can see that it is not really a sum, but a list!

Solution: Use `cp.sum(x)` instead!

Conclusion

- ▶ Modeling languages are **expressive** so that solvers can focus on being **fast**
- ▶ Modeling languages handle Boolean and integer variables, various constraints, and objectives
- ▶ CPMpy is a Python library which implements the concepts, with many different back-end solvers
- ▶ Creating and debugging models which are correct and performant is challenging and useful.

Set up your local CPMpy environment: it will be useful for the exercise sessions and to not depend on a lab PC.